Seminar 5-DSA 6.05.2025

1. Evaluate an arithmetic expression. It contains single digit operands and the +, -, *, / operators. Assume that the expression is correct.

Ex: 2 + 3 * 4 = 14 ↘ infix

Infix (operator between two operands)
2+4
4*3+6
4*(3+6)
(5+6)*(4-1)

Postfix → no paranthesis (operator after two operands)
2 4 +
4 3 * 6 +
4 3 6 + *
5 6 + 4 1 - *
9 2 +

→ We use an auxiliary queue and a stack.
→ We parse the expression for every element e.

Case 1: e is an operand (number) => push to the queue.
Case 2: e is an open paranthesis => push to the stack.
Case 3: e is a closed paranthesis => pop from the stack and push to the queue until we find open paranthesis; pop the paranthesis as well from the stack, but don't push it to the queue.
Case 4: e is an operator => while the top of the stack contains an operator with higher or equal priority than e => pop from the stack, push to the queue => push e to the stack.

- Pop everything from the stack and push to the queue.

1. Relative order of operands stays the same
. Relative order of operators might change
. No paranthesis in the postfix

Evaluate expression: 1) transform infix → postfix
2) evaluate postfix

1 x : 2*(4+3) -4 + 6/2 * (1+2*7) + 9/(1+1*4/2) + 6 + 2*8.

Queue: 2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 / 1 1 4 * 2 / + 1 + 6 + 2 8 * +

Infix → Postfix

pop from stack, push to queue

Current elem	Case	Stack ↑
2	1	
*	4	*
(	2	*(
4	1	*(4
+	3	*(+
3	4	*(+3
)	1	*
-	4	-
4	1	-
+	4	+
6	1	+
/	4	+/
2	1	+/2
*	4	+/2*
(	1	+/2*(
1	4	+/2*(1
+	1	+/2*(+
2	4	+/2*(+2
*	1	+/2*(+*
7	3	+/2*(+*7
)	4	+/2*
+	1	+/2*+
9	4	+/2*+9
/	1	+/2*+9/
(	2	+/2*+9/(
1	1	+/2*+9/(1
+	4	+/2*+9/(1+
1	1	+/2*+9/(1+1
*	4	+/2*+9/(1+1*
4	1	+/2*+9/(1+1*4
/	4	+/2*+9/(1+1*4/
2	1	+/2*+9/(1+1*4/2
)	3	+/2*+9/(1+1*4/2)
+	4	+/2*+9/(1+1*4/2)+
6	1	+/2*+9/(1+1*4/2)+6
+	4	+/2*+9/(1+1*4/2)+6+
2	1	+/2*+9/(1+1*4/2)+6+2
*	4	+/2*+9/(1+1*4/2)+6+2*
8	1	+/2*+9/(1+1*4/2)+6+2*8
	3	+/2*+9/(1+1*4/2)+6+2*8*
	4	+/2*+9/(1+1*4/2)+6+2*8*+

1. Def

```
subalgorithm transform(expr)
    init(S) // S is a stack
    init(Q) // Q is a queue        ] create stack and queue

    for e in expr execute // each elem in expr
        if e is an operand then // case 1
            push(Q,e) // queue
        else if e is ( them // case 2
            push(S,e) // stack
        else if e is ) them // case 3
            while top(S) ≠ ( execute // keep popping + pushing until open parantheses is found on the stack
                p ← pop(S) // pop from stack
                push(Q,p) // push to queue
            pop(S) // pop open paranthesis (optional)
        else // operator
            while not isEmpty(S) and top(S) ≠ ( and priority(top(S)) ≥ priority(e) execute
                ~~pop(S,e)~~ p ← pop(S) // pop from stack
                push(Q,p) // push to queue
            push(S,e) // push to stack
    while not isEmpty // expression is over, stuff left on stack
        p ← pop(S) // pop from stack
        push(Q,p) // push to queue
    transform ← Q // the queue is the final answer
```

- compare priorities in pseudocode

I. priority(e) ≥ priority(...)

II. hasHigherOrEqualPriority(e1,e2)
   → T if e1 ≥ e2 (boolean)
   → F otherwise

→ empty stack check: if expr. is not guaranteed to be correct.

as long as the stack is not empty and the top of the stack has a priority >= current operator

Evaluation:

- Use a stack
- For every element e from the queue:
  Case 1: e is an operand (number) => push to stack.
  Case 2: e is an operator => pop 2 elements from the stack, perform operation, push result to stack.
- The result is the value in the stack

1. Define an iterator for a sorted map represented as a hash table coll... ion. with separate chaining.

Queue: 2 4 3 + * ....

Current elem	stack ↑
2	2
4	2 4
3	2 4 3
+	2 7
*	14
4	~~14~~ ~~4~~
−	10
6	10 6
2	10 6 2
/	10 3
1	10 3 1
2	10 3 1 ~~2~~
7	10 3 ~~1~~ ~~2~~ ~~7~~
*	10 3 ~~1~~ ~~14~~
+	10 ~~3~~ ~~15~~
*	10 ~~45~~
+	55
9	55 9
1	55 9 1
1	55 9 1 1
4	55 9 1 ~~1~~ ~~4~~
*	55 9 1 4
2	55 9 1 ~~4~~ ~~2~~
/	55 9 ~~1~~ 2
+	55 ~~9~~ ~~3~~
/	~~55~~ ~~3~~
+	58
6	~~58~~ ~~6~~
+	64
2	64 2
8	64 ~~2~~ 8
*	64 16
+	**80**

+* → order does not matter

first popped from stack → second of the operation

↓

maintain order in non-commutative operations like 8/2.

```
function evaluate (postfix) is:
    init(S)
    while not isEmpty(postfix) execute:
        elem ← pop(postfix) // pop from queue
        if elem is operand then // number
            push(S, elem) // push to stack.
        else // operator
            op1 ← pop(S) // second operand
            op2 ← pop(S) // first operand
            res ← @compute_op2_elem_op1_
            push(S, res)
    result ← pop(S) // last value = result
    evaluate ← result
```

1. Def:

w.

E:) Pick a value and count how many values are greater than it in the arra
if K we add them together.
if >K · we need a larger treshhold
if <K we need a lower treshold

- initial value is middle between min and max.
- new treshold computed on a binary search principle.

Complexity: $O(\log_2 \text{interval} * n)$
↓
max - min

I. ITERATIVE MAX FINDING : $\Theta(K \cdot n)$
$\Theta(n)$ → find the max
do it K times

- each subsequent search must have an upper bound. (= prev max)

II. SELECTION SORT / BUBBLESORT : $\Theta(K \cdot n)$
- selection sort with descending sorting => moves the largest to the front
  => add the first K together.
- bubblesort with ascending sorting => moves the largest to the end
  => add the last K together

III. FULL SORT WITH MERGE SORT / QUICK SORT
- sorting => $\Theta(n * \log_2 n)$
- sum of first K elements : $\Theta(K)$
- total: $\Theta(n * \log_2 n) + \Theta(K) \in \Theta(n * \log_2 n)$

IV. K-SIZED ARRAY (SORTED) : $O(n \cdot K)$
- aux sorted array of K largest elems seen so far
- compare new elems with array's min
  → if > min => update min
  => re-sort the array

V. K-SIZED ARRAY (UNSORTED): $O(n \cdot K)$
- aux unsorted array
- compare new elems with array's min
  → if > min => update min

VI. BINARY MAX-HEAP (n elems) : $O(n * \log_2 n) + \Theta(K * \log_2 n) \xrightarrow{n \geq K} O(n * \log_2 n)$
- add al elements to a heap and remove the first K elems.
  → add: $O(\log_2 n)$; → remove: $\Theta(\log_2 n)$

VII. BINARY MIN-HEAP (K elems): $O(n \log_2 K$
- min-heap with K elems, if elem > heap's root => replace the root
  => re-heapify 2

VIII. OPTIMISED HEAP (W ARRAY)
~ V but sum at the end instead of removing one by one => $O(n+K \log_2$

IX. MAX-HEAPIFY $O(n + K \log_2 n)$
- array => heap ; remove root K times

2. Determine the sum of the largest k elements from a vector containing n distinct numbers.

n=10 : [6, 12, 9, 91, 3, 5, 25, 81, 11, 23] k=3 => 91 + 81 + 25 = 172+25 = 197.

1) Choose a random pivot and rearrange the array so that the elements smaller than the pivot are in front of it, elements greater are after.
Assume pivot goes to position i
i-1 elements smaller than the pivot.
n-i elements greater than the pivot

1 2 3 4
2 1

if $k \leq n-i$ we only care for the second half. => we recursively find the largest k elements from it.
if $k = n-i$ => result is sum of elements from second half.
if $k = n-i+1$ => result is ... + pivot.
if $k > n-i+1$ => result is the sum of elements from the second half + pivot + sum of largest $k-(n-i+1)$ elements from the first half. (recursive call)

n=10 : [6, 12, 9, 91, 3, 5, 25, 81, 11, 23] k = 3
1 2 3 4 5 6 7 8 9 10
[6, 12, 9, 23, 3, 5, 11, 25, 81, 91]
< >
-> BEST CASE.

k=5 : 25 + 81 + 91 +

k=2 : [6, 12, 9, 23, 3, 5, 11]
1 2 3 4 5 6 7
[6, 3, 5, 9, 12, 23, 11]
ignore what we need.

k=2 : [12, 23, 11]
[11, 12, 23]
what we need

Complexity: BC: $\Theta(n)$
WC: $\Theta(n^2)$
TC: $O(n^2)$